

New Shop Onboarding

Step-by-step playbook for setting up a new shop (e.g. *Balamurugan*) on the pawnbroking cloud + Magizhchi Share pipeline.

Phase A — Cloud setup (admin, ~5 minutes)

Done once from any computer with Railway access. Substitute **balamurugan** with the real shop_id throughout.

A1. Add the shop as a tenant

Option 1 (easy) — redeploy with the shop in env var. Railway → cloud-api service → Variables → set:

```
TENANTS=alwarpuram,annanagar,mylocal,balamurugan
```

Redeploy. On boot the log prints:

```
provisioning tenant schema 'balamurugan'
API KEY for shop 'balamurugan' = BALAMURUGAN_XXXXXXXXXXXXXXXXX <-- SAVE THIS
```

Option 2 (SQL only, no redeploy) — run on Railway Postgres:

```
INSERT INTO public.tenants(shop_id, schema_name, display_name, primary_email)
VALUES ('balamurugan', 'balamurugan', 'Balamurugan',
      'balamurugan_owner@gmail.com');
```

```
CREATE SCHEMA IF NOT EXISTS balamurugan;
-- then run the contents of db/tenant/tenant.sql with
-- search_path = balamurugan, public to provision tables.
```

```
INSERT INTO public.shop_credentials(api_key, shop_id, label)
VALUES ('BALAMURUGAN_' ||
      replace(gen_random_uuid()::text, '-', ''),
      'balamurugan', 'desktop agent')
RETURNING api_key; -- SAVE THE PRINTED KEY
```

A2. Set the primary email

The mobile app for balamurugan can only be signed in with this email. Set it if you didn't above:

```
UPDATE public.tenants
SET primary_email = 'balamurugan_owner@gmail.com'
WHERE shop_id = 'balamurugan';
```

Phase B — Owner registers on Magizhchi Share (~3 min)

Gives them a Magizhchi account so the cloud can later store bills, images and backup files on their behalf.

- Open <https://box.magizhchi.software> in any browser.
- Sign up with mobile + email = **balamurugan_owner@gmail.com** + display name.
- Verify the OTP they receive.

Don't issue API tokens manually. The cloud auto-mints one on first OTP login from the phone (Phase D2).

Phase C — Shop PC setup (Windows, ~15 min)

C1. PostgreSQL

- Install Postgres 18 for Windows. Default port 5432, set a password.

- Create the database:

```
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -c "CREATE DATABASE pawnbroking"
```

- Restore the desktop app's schema/data from your master SQL dump.

C2. Java 17+

- Install Eclipse Temurin JDK 17 (or 21). Verify:

```
java -version
```

C3. Desktop pawnbroking app

Install as usual. On first run it'll seed CMP1 in the company table. The user can change/add companies later via the desktop UI.

C4. Sync agent (Windows service)

- Copy pawnbroking-sync-agent-1.0.0-shaded.jar, pawnbroking-sync.exe (WinSW renamed), and pawnbroking-sync.xml to C:\PawnSync\.

- Create C:\ProgramData\PawnBroking\sync.properties:

```
db.url=jdbc:postgresql://localhost:5432/pawnbroking
db.user=postgres
db.password=<their postgres password>
```

```
cloud.url=https://devpawn.magizhchi.academy
cloud.api_key=<paste BALAMURUGAN_xxx api_key from Phase A1>
shop.id=balamurugan
```

```
batch.size=200
poll.interval.ms=5000
health.port=17654
schema.guard.interval.ms=30000
image.scan.interval.ms=60000
```

- Install + start the service (elevated PowerShell):

```
cd C:\PawnSync
.\pawnbroking-sync.exe install
.\pawnbroking-sync.exe start
```

- Health check:

```
curl http://127.0.0.1:17654/health # should return OK
```

SchemaGuard installs sync_outbox, triggers, sync_image_uploads, and sync_backup_uploads automatically on first run.

C5. Backfill historical data (only if the desktop already has bills)

On the shop's local Postgres:

```
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -d pawnbroking \
-f "C:\path\to\pawnbroking-outbox\migrations\BACKFILL__existing_rows.sql"
```

Agent picks up new outbox rows within 5 s. At box rate-limit (60 req/min), ~1000 bills + their images take ~17 min to fully sync.

Phase D — Owner's phone (~2 min)

D1. Build the per-shop APK

SHOP_ID is hardcoded per build. Edit one line in

PawnAndroidApp/app/src/main/java/com/pawnbroking/app/config/AppConfig.java:

```
public static final String SHOP_ID = "balamurugan";
```

Build:

```
cd D:\Pawnbroking\PawnBrokingMobApp\PawnAndroidApp
.\gradlew.bat assembleDebug
```

APK lands at app\build\outputs\apk\debug\app-debug.apk. Send via Gmail / WhatsApp / USB.

D2. Install + first login (auto-mints the Magizhchi token)

- Owner installs APK (allow 'install unknown apps').
- Email **balamurugan_owner@gmail.com** → **Send OTP** → enter 6-digit code → **Verify & Sign In**.
- Behind the scenes (no action needed):
 - Cloud-api receives box's accessToken from OTP verify.
 - Cloud-api calls POST <https://box.magizhchi.software/api/api-tokens> with that JWT.
 - Box returns an mbk_... token + the user's personal drive id.
 - Cloud-api stores both in public.tenants for balamurugan.

D3. Verify everything's wired

From your laptop, on Railway Postgres:

```
SELECT shop_id, primary_email,
       magizhchi_token IS NOT NULL AS has_box_token,
       magizhchi_conversation_id
FROM public.tenants WHERE shop_id='balamurugan';
```

Expected: row exists, primary_email set, has_box_token = true, conv_id populated.

Phase E — Backup folder sync (automatic)

From this build forward, the sync-agent **also** uploads every file in the shop's `company.backup_file_path` folder to the Magizhchi box under `backups/<companyId>/<relative-path>/`.

- No extra config — the column is read from the local company table every scan tick (default 60 s).
- Tracked in local `sync_backup_uploads` table so each file uploads only once. Re-running the desktop's backup procedure to overwrite a file will re-upload (the cloud upserts under the same key).
- Subject to the same 60 req/min box rate limit as bill images.

If `company.backup_file_path` is blank or the directory doesn't exist, the watcher silently skips it — no errors.

Phase F — That's it

- Shop owner uses the desktop app normally.
- Every new bill / customer / image syncs to cloud in ~5 s (events) and ~1 min (images / backup files).
- Phone shows real-time data + bell notifications, charts refresh per company.

Reference — Files shipped to each new shop's PC

File	From your repo	Lands on shop PC
pawnbroking-sync-agent-1.0.0-shaded.jar	pawnbroking-sync-agent/target/	C:\PawnSync\
pawnbroking-sync.exe (WinSW renamed)	pawnbroking-sync-agent/winsw/	C:\PawnSync\
pawnbroking-sync.xml	pawnbroking-sync-agent/winsw/	C:\PawnSync\
sync.properties (per-shop)	template: pawnbroking-sync-agent/sync.properties	C:\ProgramData\PawnBroking\sync.properties
BACKFILL__existing_rows.sql	pawnbroking-outbox/migrations/	run locally via psql
app-debug.apk (per-shop, edit SHOP_ID first)	PlayAndroidApp/app/build/outputs/apk/debug/	upload to owner's Android phone

Future improvement — one APK for every shop

Today you build a separate APK per shop because SHOP_ID is hardcoded. A cleaner model: the Login screen asks for email first, cloud-api tells the app which shop that email belongs to (using the existing primary_email column), and the app stores it locally for the session. Then one APK works for every shop forever and onboarding skips Phase D1 entirely.